

PROJET ADOMC

MIN-CONFLICTS : UNE RÉVOLUTION DANS LA RÉSOLUTION DE CSP

Présenté par :

2768 - ANDRIAMANANTENA Elvis Sylvano

2754 - SAMUELSON RAHITAVOLA Chryswdell
Jeannioh

MAI 2026

2738 - TIDA Ratsaranjanahary Loralien

2587 - RABEARIMANANA Faniriniaina Luciano

INTRODUCTION AUX CSP

Un Problème de Satisfaction de Contraintes (CSP) est défini par un ensemble de variables, de domaines et de contraintes à satisfaire simultanément.

C'est un pilier fondamental de l'optimisation combinatoire et de l'intelligence artificielle décisionnelle.



Planification d'horaires



Allocation de ressources



Jeux (Sudoku, N-Reines)



Configuration industrielle

COMPLEXITÉ : SOUVENT NP-COMPLET

QU'EST-CE QUE MIN-CONFLICTS ?

C'est une **méta-heuristique de recherche locale** pour les CSP. Contrairement aux approches systématiques, elle privilégie la rapidité et l'efficacité sur des problèmes de grande taille.

NE PAS CONFONDRE AVEC LE BACKTRACKING

AFFECTATION COMPLÈTE

On part d'un état où toutes les variables ont une valeur (souvent aléatoire ou gloutonne).

RÉPARATION ITÉRATIVE

L'algorithme modifie les valeurs pour réduire progressivement le nombre de conflits.

OPTIMISATION LOCALE

Chaque étape vise à améliorer localement l'état actuel sans explorer tout l'arbre de recherche.

IMPLÉMENTATION PYTHON

```
import random

class NReines:
    def __init__(self, n):
        self.n = n

    def conflits(self, var, val, assign):
        cnt = 0
        for ligne, col in assign.items():
            if ligne == var: continue
            if col == val or abs(ligne - var) == abs(col - val):
                cnt += 1
        return cnt

    def total_conflits(self, assign):
        return sum(self.conflits(v, assign[v], assign) for v in assign)

def min_conflict_steps(n, max_steps=500):
    csp = NReines(n)
    assign = {v: random.randrange(n) for v in range(n)}
    tc = csp.total_conflits(assign)
    angry = [v for v in assign if csp.conflits(v, assign[v], assign) > 0]

    for step in range(1, max_steps + 1):
        if tc == 0: break
        var = random.choice(angry)
        best = float('inf')
        best_vals = []
        for col in range(n):
            c = csp.conflits(var, col, assign)
            if c < best:
                best = c
                best_vals = [col]
            elif c == best:
                best_vals.append(col)
```

CLASSE NREINES

Définit la logique des conflits (colonnes et diagonales) pour le problème des N-Reines.

INITIALISATION

L'état initial est généré aléatoirement (`randrange`). On identifie les variables "angry" (en conflit).

BOUCLE PRINCIPALE

L'algorithme itère jusqu'à la solution (`tc == 0`) ou la limite `max\_steps`.

CHOIX OPTIMAL

Pour une variable en conflit, on teste toutes les colonnes et on choisit celle minimisant les **conflits locaux**.

ALÉATOIRE

En cas d'égalité de conflits (`best\_vals`), un choix aléatoire est fait pour éviter les minima locaux.

LES 3 POINTS CLÉS

01

VARIABLES EN CONFLIT

L'algorithme cible uniquement les variables qui **violent au moins une contrainte**, réduisant l'espace de recherche.

02

MINIMISATION LOCALE

On choisit la valeur qui **minimise les conflits**, en adoptant une réparation itérative.

03

ASPECT STOCHASTIQUE

Le **tirage aléatoire** prévient le blocage dans des minima locaux et améliore la robustesse.

EFFICACITÉ : L'EXEMPLE DES N- REINES

ORIGINE HISTORIQUE

Découvert en 1990 par Minton et al. pour l'ordonnancement complexe du **télescope Hubble**.

PERFORMANCE N-REINES

Pour **256 de reines**, environ 150 réaffectations suffisent. Le temps de résolution est quasi-linéaire.

TAILLE	BACKTRACKING	MIN-CONFLICTS
10 reines	~ 1 min	< 0.1 sec
100 reines	Impossible	< 17 sec
256 reines	Impossible	~ 38 sec

COMPARAISON DES TEMPS DE RÉOLUTION

FORCES

AVANTAGES CLÉS

ULTRA-RAPIDE

Très performant sur problèmes denses.

PASSAGE À L'ÉCHELLE

Gère des millions de variables efficacement.

SIMPLICITÉ

Algorithme léger, facile à implémenter.

FAIBLESSES

LIMITES

INCOMPLÉTUDE

Ne garantit pas de trouver une solution.

SENSIBILITÉ

Dépend fortement de l'état initial.

STRUCTURES COMPLEXES

Échec sur problèmes mal structurés.

PROJET ADOMC : VISUALISATION

Notre application rend l'algorithme **Min-Conflicts** visible et compréhensible à travers une interface moderne et interactive.

Elle permet d'observer en temps réel comment l'algorithme "répare" le problème des **N-Reines**, passant d'un chaos initial à une solution parfaite.



GRILLE DYNAMIQUE

Configuration personnalisée (ex: 20×20) pour tester différentes échelles.



TIMELINE INTERACTIVE

Navigation pas à pas dans les itérations de l'algorithme.



STATS EN DIRECT

Suivi visuel de la diminution des conflits à chaque étape.

STACK & OUTILS

PYTHON & FLASK

Logique CSP robuste et API REST performante.

JS & PARTICLES.JS

Interface dynamique et visualisation fluide.

STATS TEMPS RÉEL

Analyse itérative et timeline de résolution.

BENCHMARK N-REINES

N	Succès %	Temps (ms)	Étapes
8	80.0	1.33	34.5
16	80.0	6.79	88.6
32	100.0	22.21	100.2
64	100.0	82.07	96.5
128	100.0	491.34	145.5
256	10.0	2664.31	183.0

DÉMONSTRATION



CONFIGURATION

« Voici l'interface **N-Reines 20×20**. On observe un état initial généré aléatoirement avec de nombreux conflits. »



LANCEMENT

« Lançons la résolution. L'algorithme choisit itérativement la **reine la plus attaquée**... »



RÉPARATION

« ...et la déplace sur la case avec le **minimum de conflits**. On voit les conflits diminuer visuellement. »



ANALYSE TIMELINE

« Grâce à la timeline, nous pouvons naviguer dans chaque étape du **for i ← 1 to max_steps**. »

CONCLUSION

« Min-Conflicts a révolutionné la résolution de CSP en passant d'une approche systématique à une réparation locale stochastique. »

VISUALISATION

Notre application rend l'algorithme visible et compréhensible pour tous.

OBJECTIF PÉDAGOGIQUE

Transformer des concepts d'IA abstraits en une expérience concrète et interactive.